

Proofs are Programs

Structured Data

Short Recap

Proofs by

Case Analysis

```
destruct n as [ | n'] eqn:E.
```

Induction

```
induction n as [ | [n' H]].
```

Introduction Patterns

```
destruct c as [ | | p ].
```

```
intros [ | | [] ].
```

“Informal Proofs”



Proof: by induction on n.

Proofs within Proofs

```
replace ... with ...  
{ ... }
```

Pairs of Numbers

Lists (of Natural Numbers)

```
Inductive natlist : Type :=  
  | nil  
  | cons (n : nat) (l : natlist).
```

Lists (of Natural Numbers)

```
Inductive natlist : Type :=  
  | nil  
  | cons (n : nat) (l : natlist).
```

```
cons 1 (cons 2 (cons 3 nil))
```

Lists (of Natural Numbers)

```
Inductive natlist : Type :=  
  | nil  
  | cons (n : nat) (l : natlist).
```

```
cons 1 (cons 2 (cons 3 nil))
```

```
1 :: (2 :: (3 :: nil))
```


Lists (of Natural Numbers)

```
Inductive natlist : Type :=  
  | nil  
  | cons (n : nat) (l : natlist).
```

```
cons 1 (cons 2 (cons 3 nil))
```

```
1 :: (2 :: (3 :: nil))
```

```
1 :: 2 :: 3 :: nil
```

Lists (of Natural Numbers)

```
Inductive natlist : Type :=  
  | nil  
  | cons (n : nat) (l : natlist).
```

```
cons 1 (cons 2 (cons 3 nil))
```

```
1 :: (2 :: (3 :: nil))
```

```
1 :: 2 :: 3 :: nil
```

```
[1;2;3]
```


Lists (of Natural Numbers)

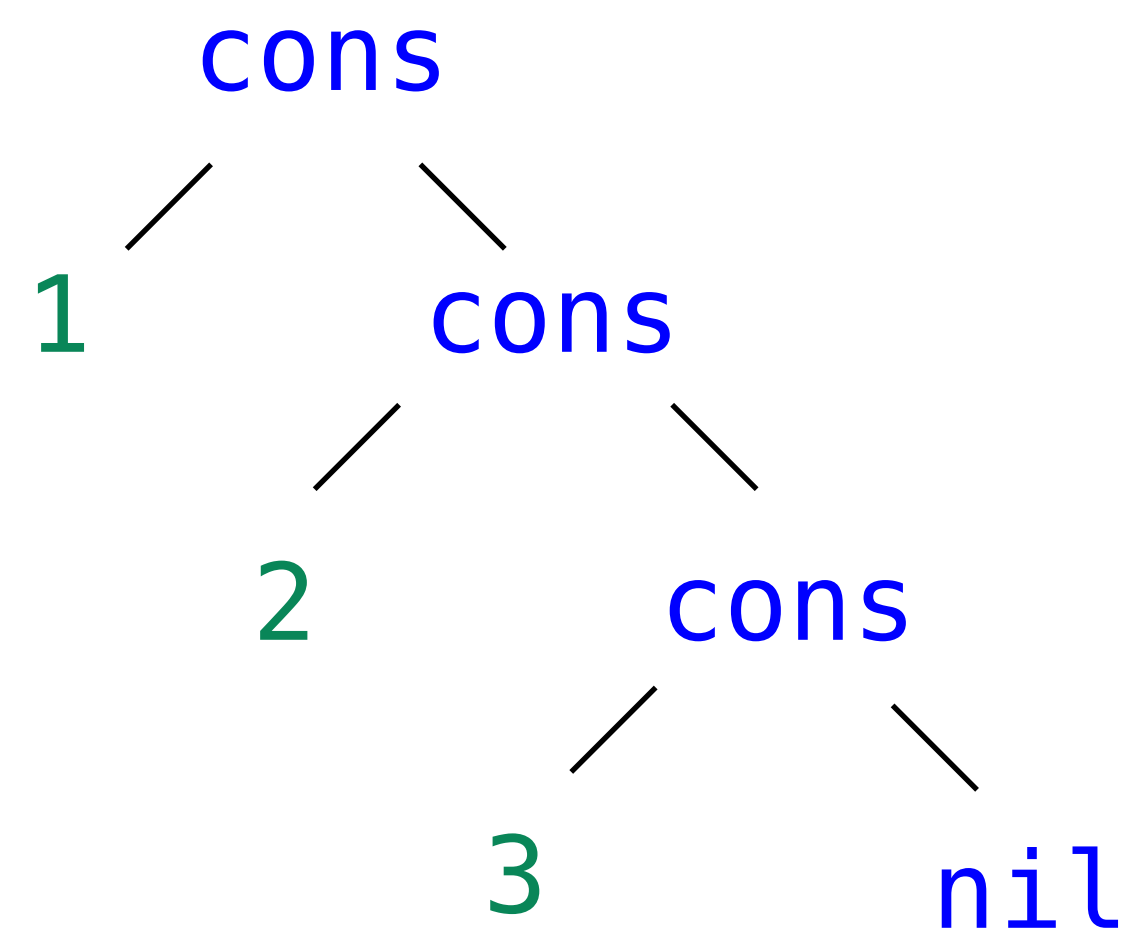
```
Inductive natlist : Type :=  
  | nil  
  | cons (n : nat) (l : natlist).
```

```
cons 1 (cons 2 (cons 3 nil))
```

```
1 :: (2 :: (3 :: nil))
```

```
1 :: 2 :: 3 :: nil
```

```
[1;2;3]
```



Lists (of Natural Numbers)

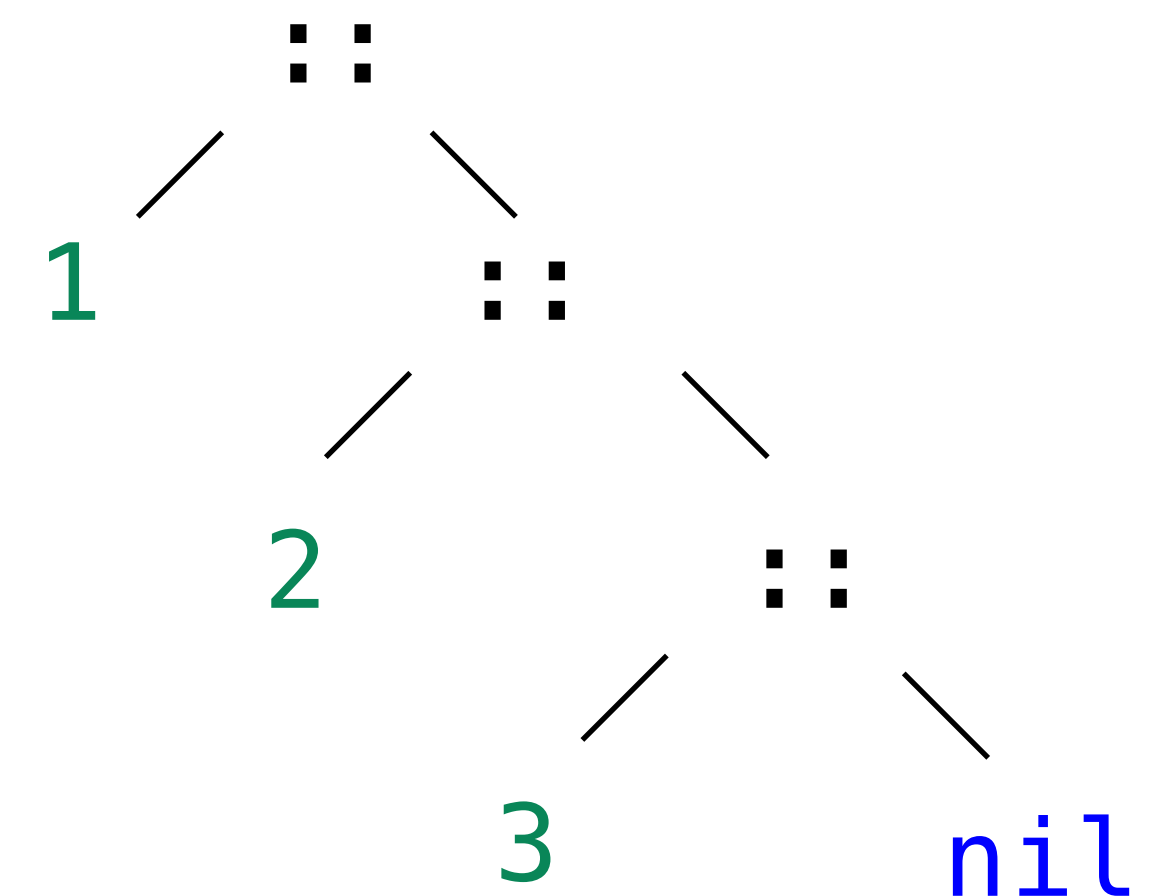
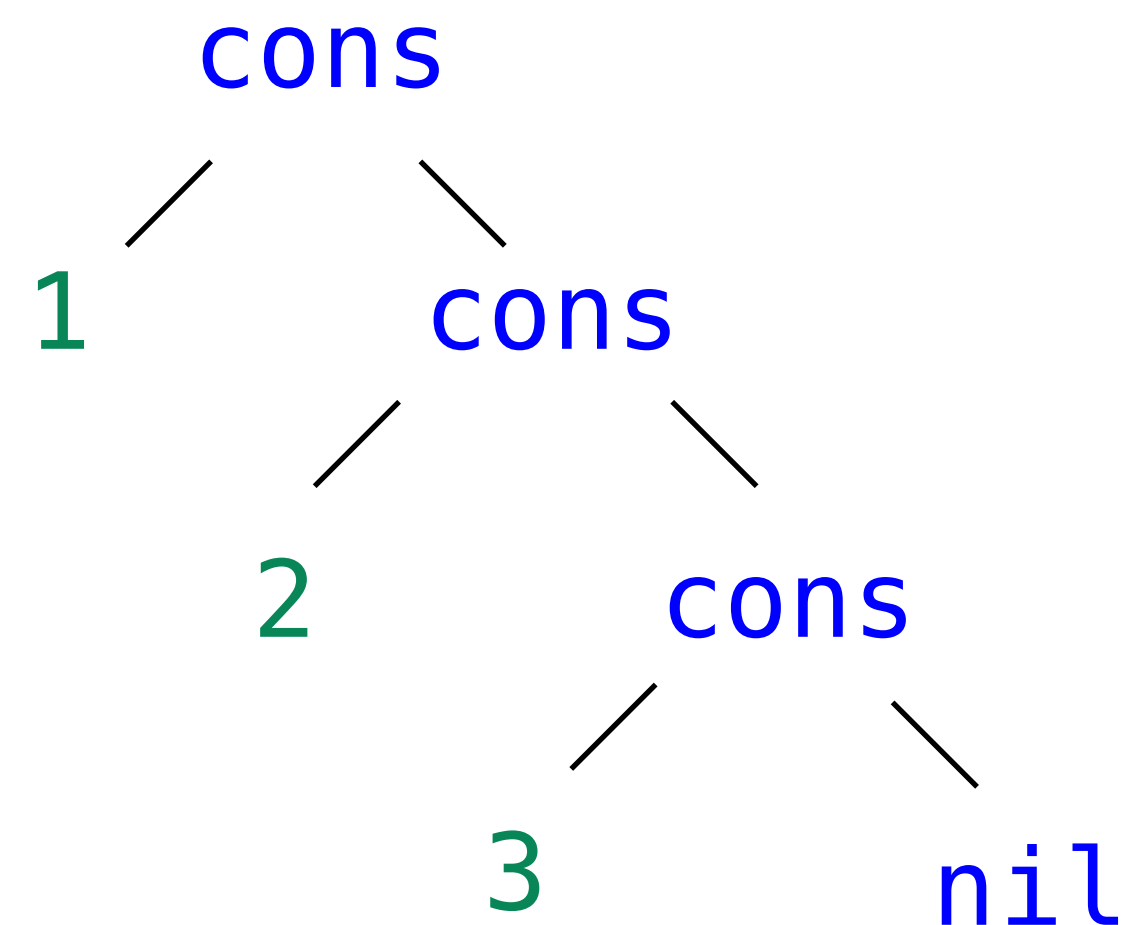
```
Inductive natlist : Type :=  
  | nil  
  | cons (n : nat) (l : natlist).
```

```
cons 1 (cons 2 (cons 3 nil))
```

```
1 :: (2 :: (3 :: nil))
```

```
1 :: 2 :: 3 :: nil
```

```
[1;2;3]
```



Building Lists Recursively

repeat 42 2

```
Fixpoint repeat (n count : nat) : natlist :=  
  match count with  
  | 0 => nil  
  | S count' => n :: (repeat n count')  
  end.
```


Building Lists Recursively

repeat 42 2

repeat 42 (S (S 0))

```
Fixpoint repeat (n count : nat) : natlist :=  
  match count with  
  | 0 => nil  
  | S count' => n :: (repeat n count')  
end.
```

Building Lists Recursively

repeat 42 2

repeat 42 (S (S 0))



42

::

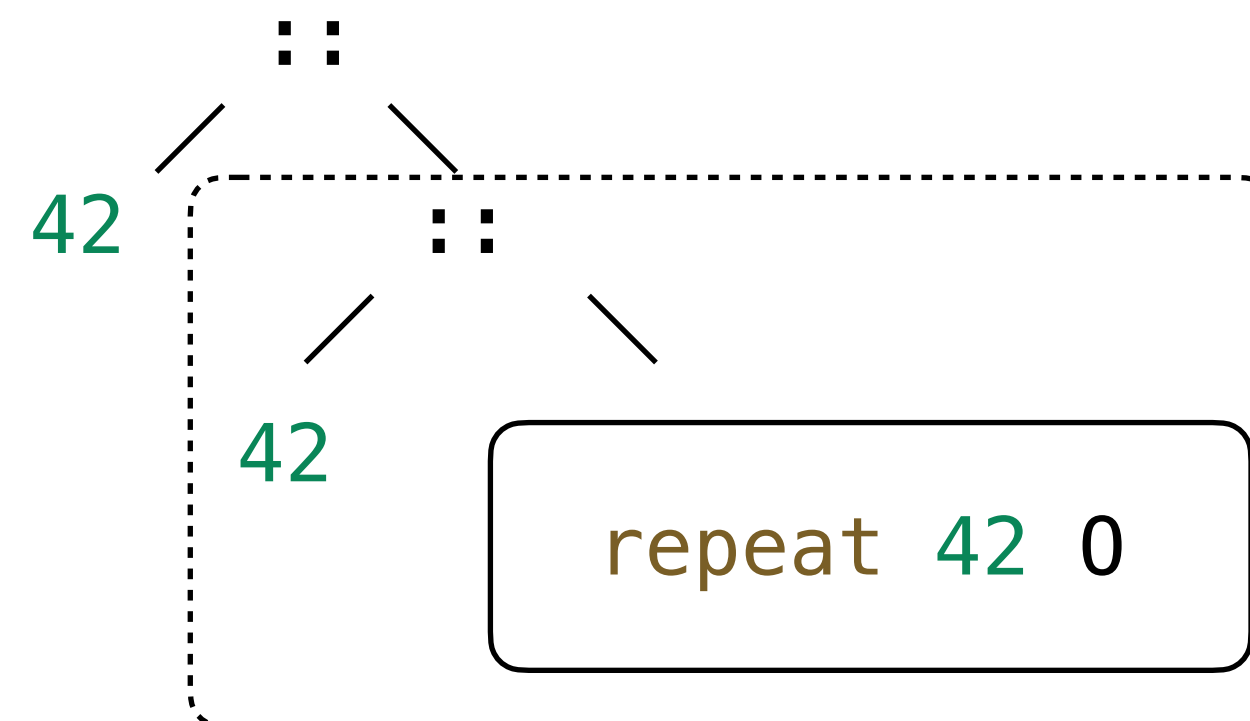
repeat 42 (S 0)

```
Fixpoint repeat (n count : nat) : natlist :=  
  match count with  
  | 0 => nil  
  | S count' => n :: (repeat n count')  
end.
```

Building Lists Recursively

repeat 42 2

repeat 42 (S (S 0))



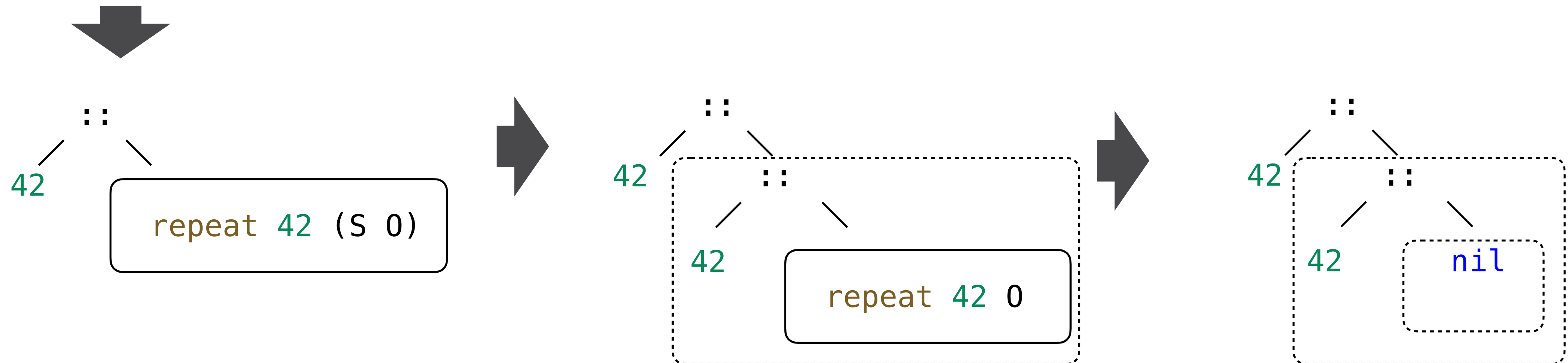
```
Fixpoint repeat (n count : nat) : natlist :=  
  match count with  
  | 0 => nil  
  | S count' => n :: (repeat n count')  
end.
```


Building Lists Recursively

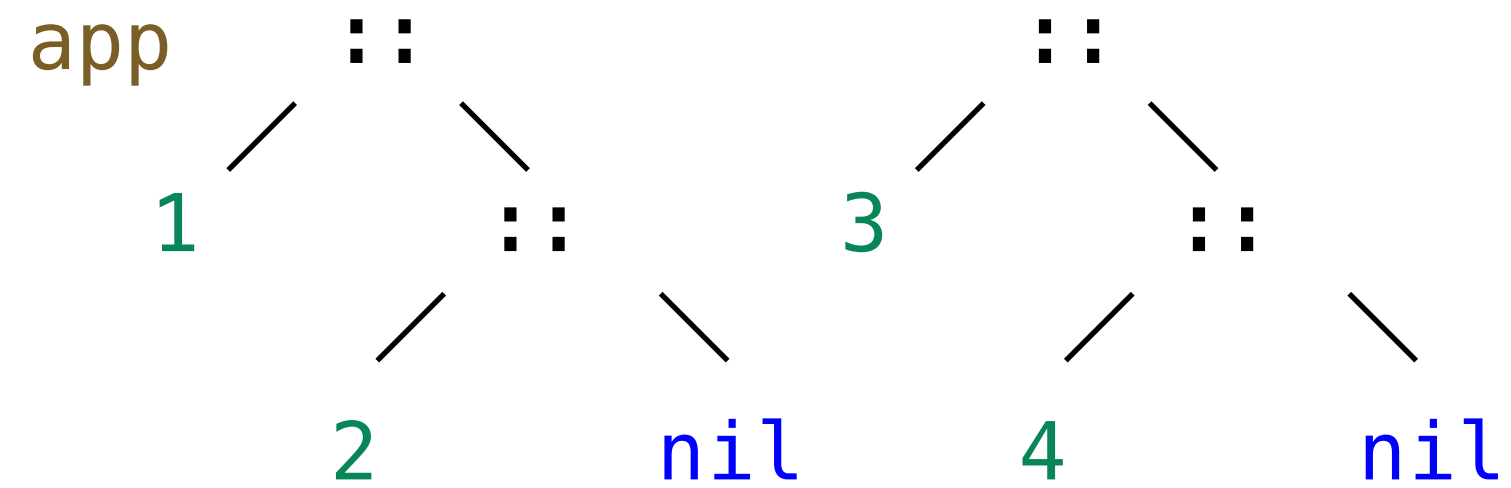
repeat 42 2

repeat 42 (S (S 0))

```
Fixpoint repeat (n count : nat) : natlist :=  
  match count with  
  | 0 => nil  
  | S count' => n :: (repeat n count')  
  end.
```

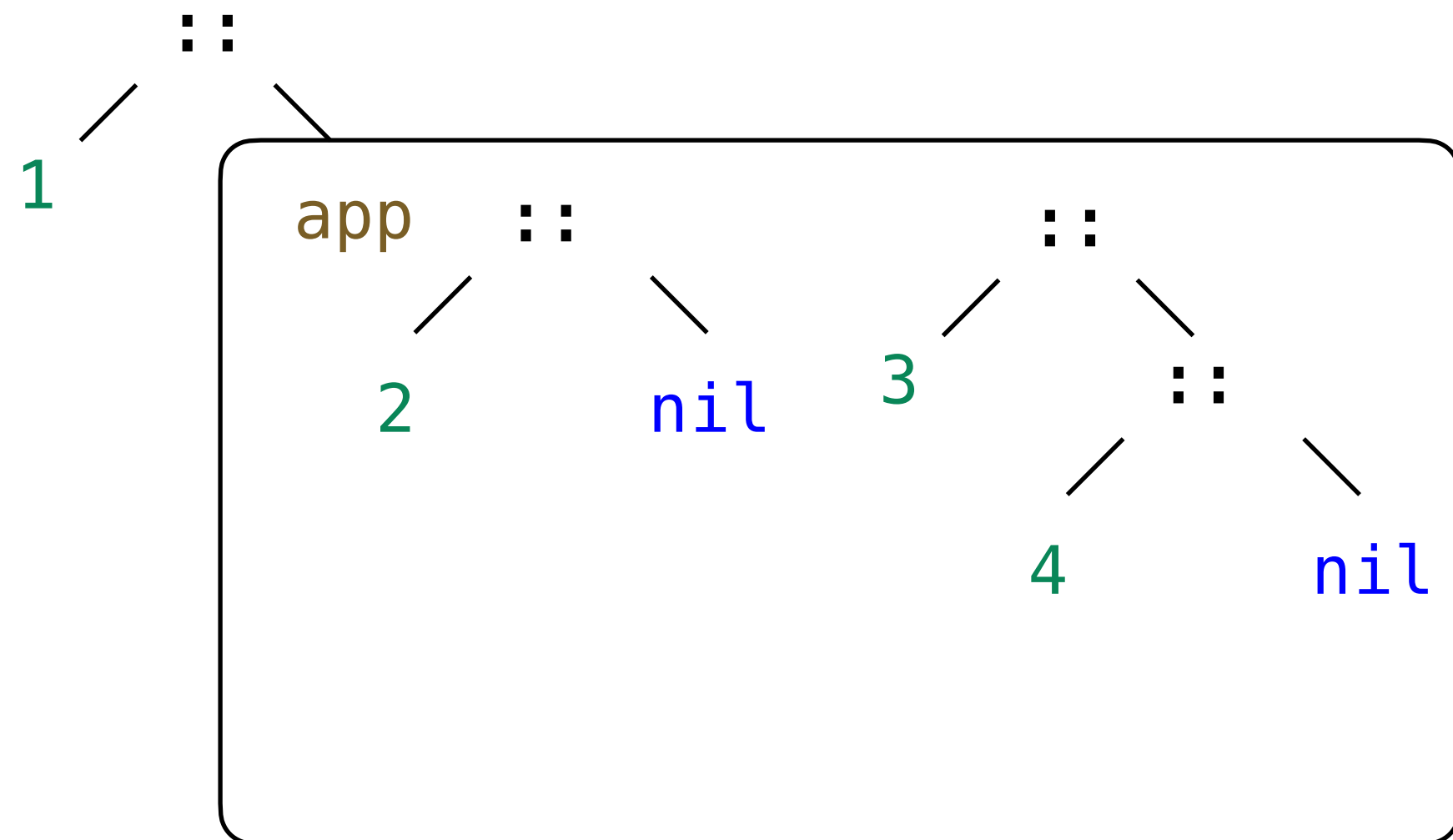
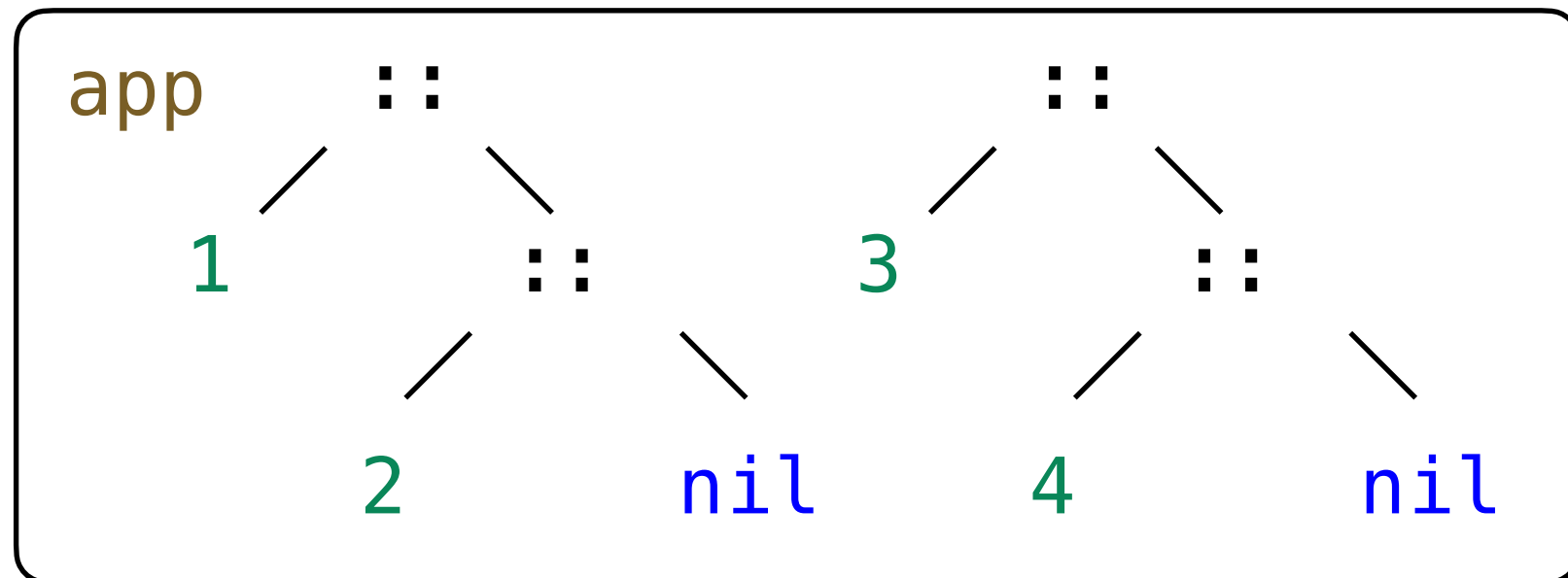


Recursive Functions on Lists



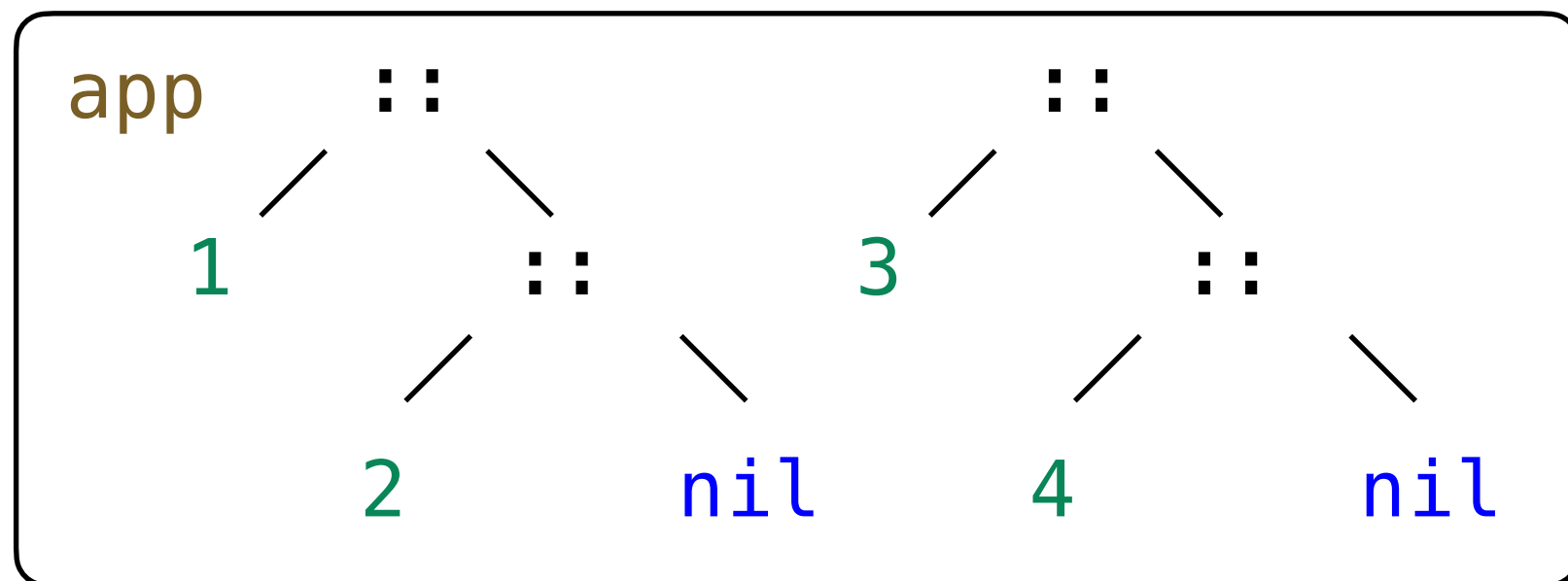
```
Fixpoint app (l1 l2 : natlist) : natlist :=  
  match l1 with  
  | nil      => l2  
  | h :: t   => h :: (app t l2)  
end.
```


Recursive Functions on Lists

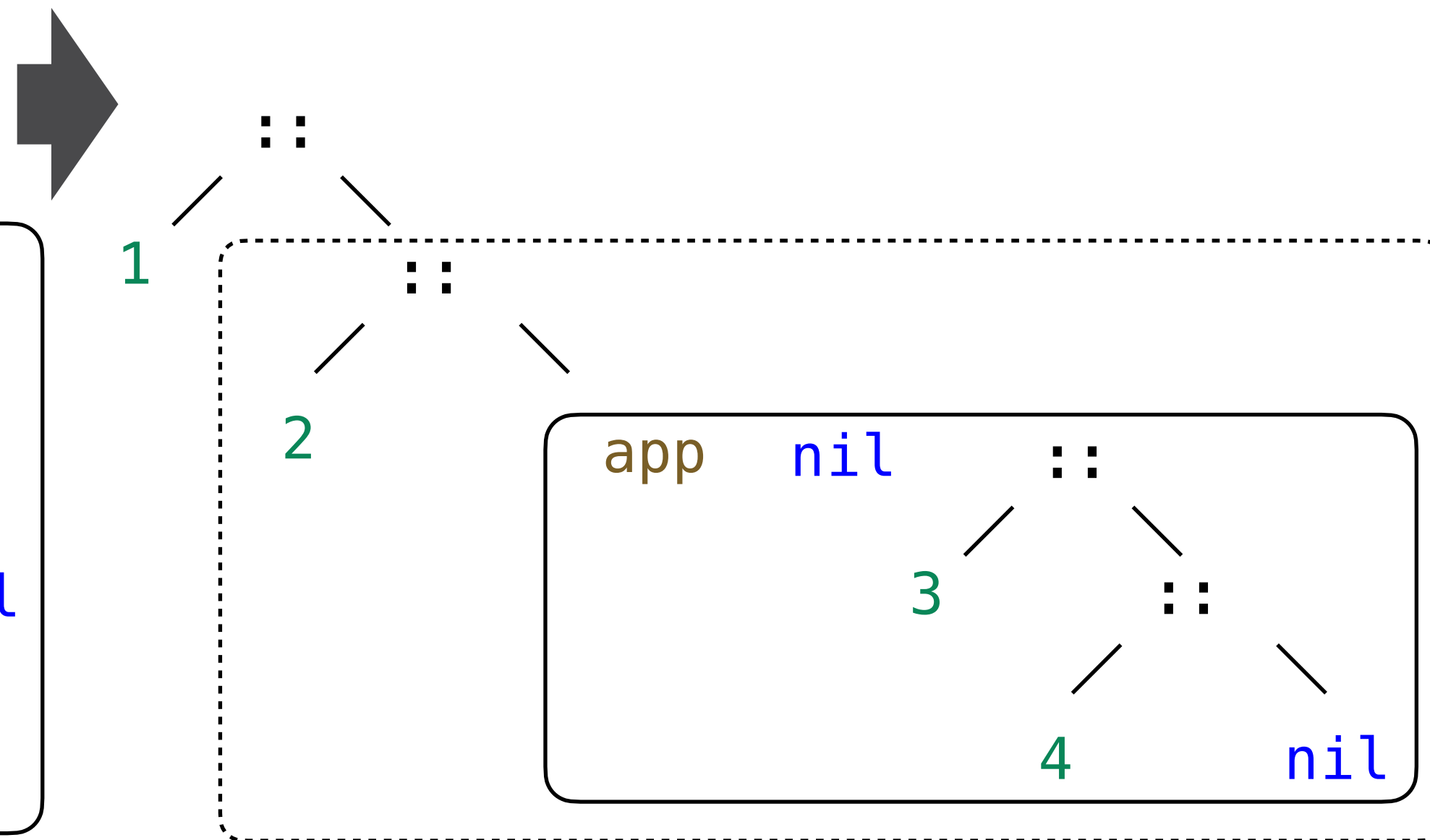
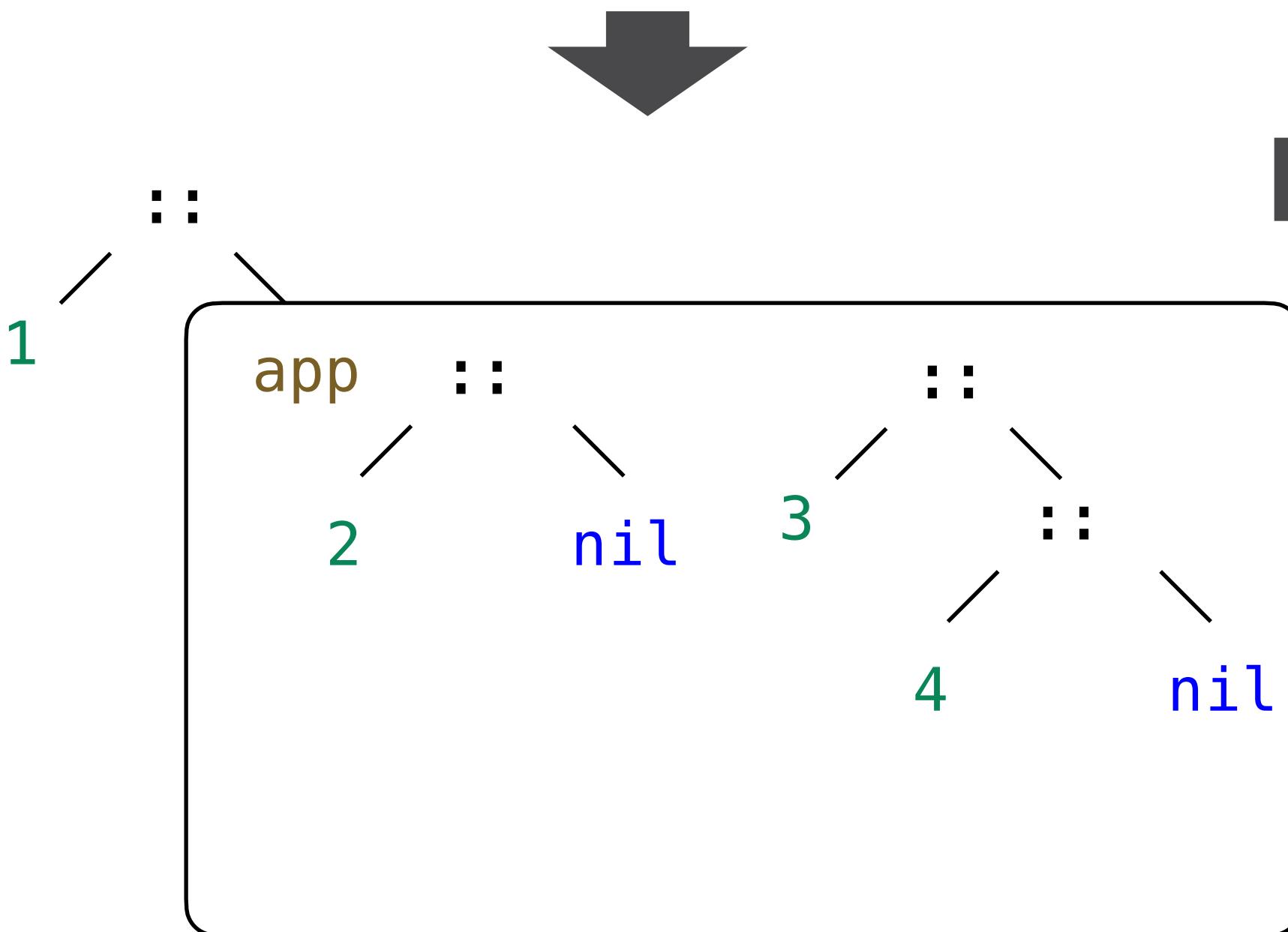


```
Fixpoint app (l1 l2 : natlist) : natlist :=  
  match l1 with  
  | nil    => l2  
  | h :: t => h :: (app t l2)  
end.
```

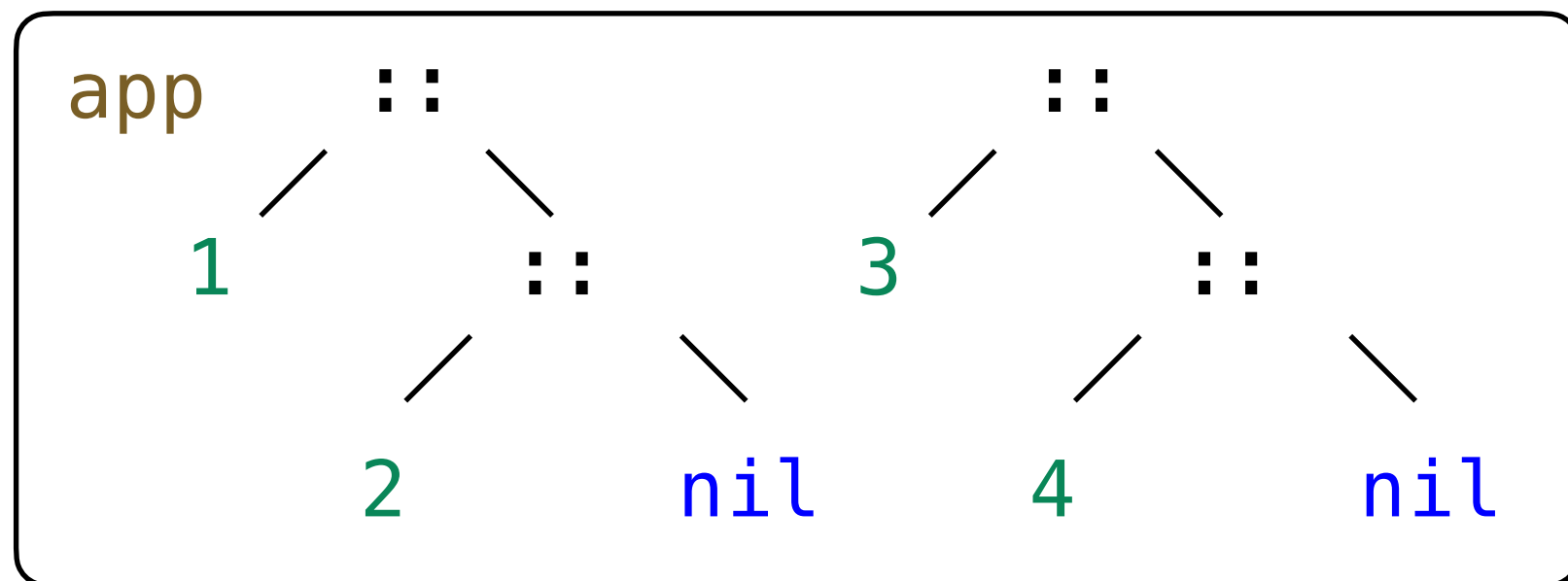
Recursive Functions on Lists



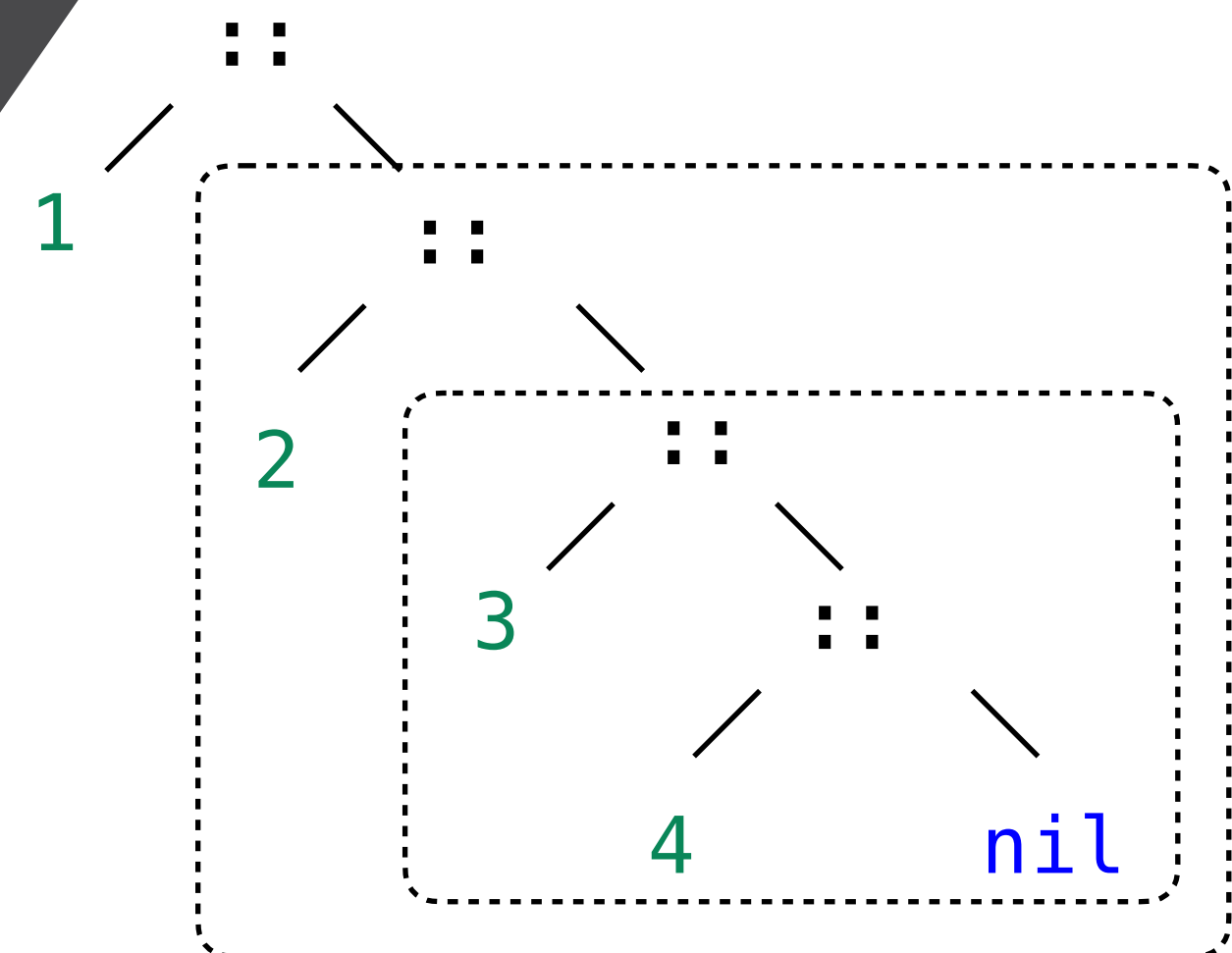
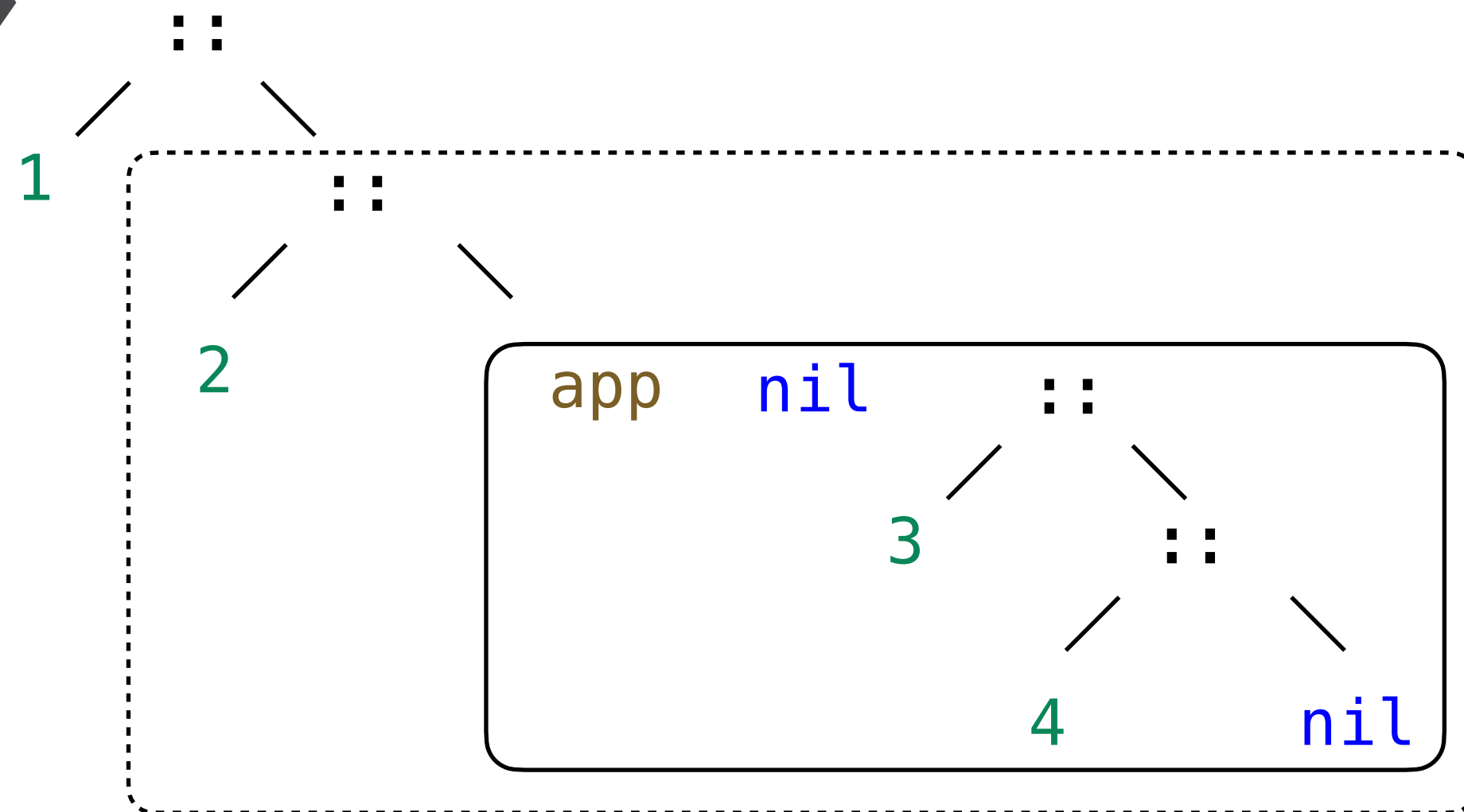
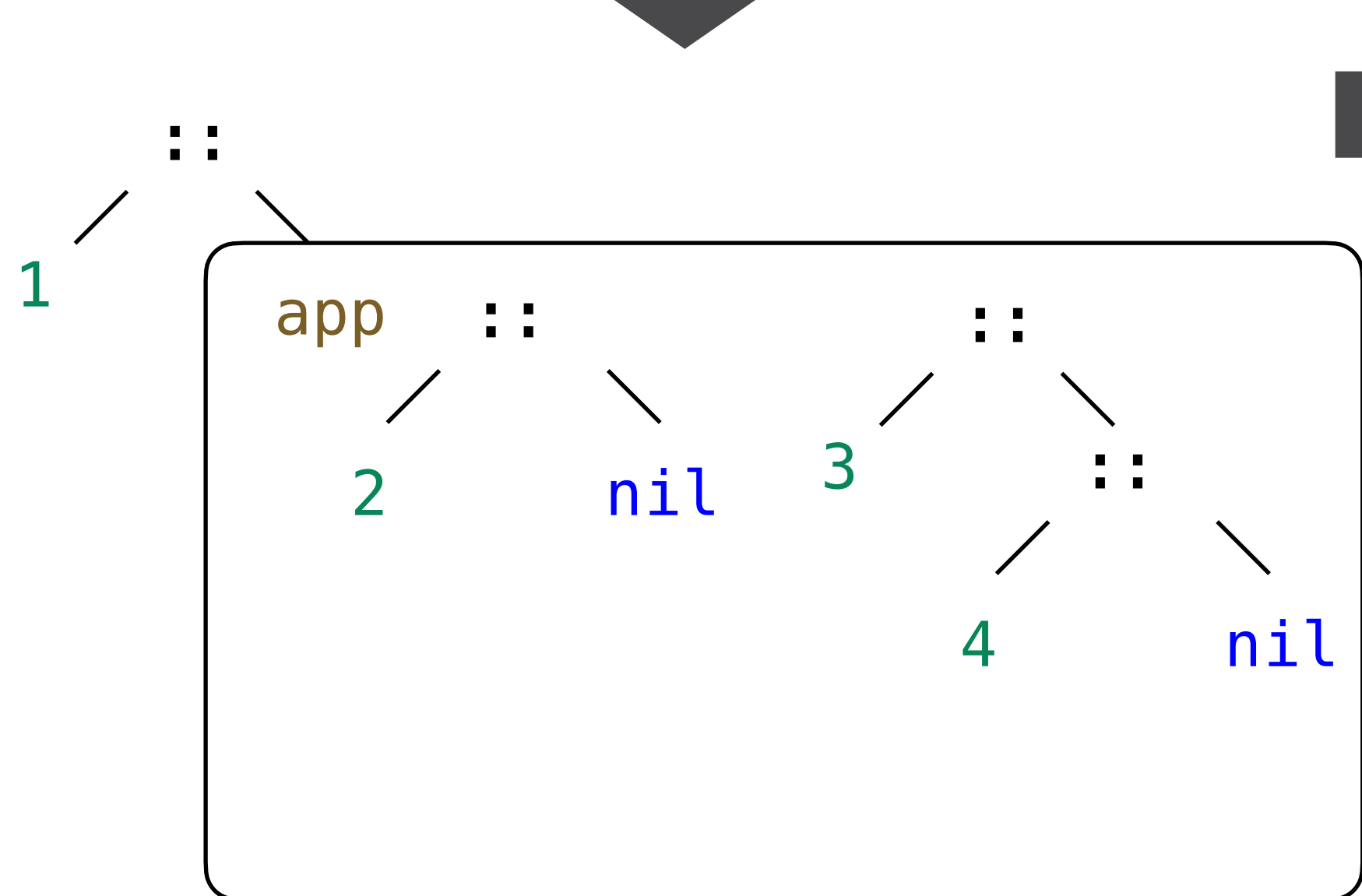
```
Fixpoint app (l1 l2 : natlist) : natlist :=  
  match l1 with  
  | nil    => l2  
  | h :: t => h :: (app t l2)  
end.
```



Recursive Functions on Lists



```
Fixpoint app (l1 l2 : natlist) : natlist :=  
  match l1 with  
  | nil    => l2  
  | h :: t => h :: (app t l2)  
end.
```





<https://etc.ch/HCCs>

Quiz

```
Fixpoint foo (n : nat) : natlist :=  
  match n with  
  | 0 => nil  
  | S n' => n :: (foo n')  
end.
```

Induction on Lists

- General principle to show that $\forall \ell . P(\ell)$
 - show $P(\text{nil})$
 - show that for any $\ell = x :: \ell'$ if $P(\ell')$ holds, then so does $P(\ell)$
- Example: $P(\ell) := \ell ++ \text{nil} = \ell$
- Example: $P(\ell_1) := \forall \ell_2 \ell_3 . (\ell_1 ++ \ell_2) ++ \ell_3 = \ell_1 ++ (\ell_2 ++ \ell_3)$

Example: Induction on Lists

$$\forall \ell_1 \ell_2 \ell_3. (\ell_1 ++ \ell_2) ++ \ell_3 = \ell_1 ++ (\ell_2 ++ \ell_3)$$

```
Fixpoint app (l1 l2 : natlist) : natlist :=  
  match l1 with  
  | nil      => l2  
  | h :: t   => h :: (app t l2)  
  end.
```


Example: Induction on Lists

- To show: $\forall \ell_1 \ell_2 \ell_3. (\ell_1 ++ \ell_2) ++ \ell_3 = \ell_1 ++ (\ell_2 ++ \ell_3)$
- Proof: By induction on ℓ_1
 - Suppose that $\ell_1 = []$.
We show: $([] ++ \ell_2) ++ \ell_3 = [] ++ (\ell_2 ++ \ell_3)$
This holds as: $([] ++ \ell_2) ++ \ell_3 = \ell_2 ++ \ell_3 = [] ++ (\ell_2 ++ \ell_3)$ by definition of $++$
 - Suppose that $\ell_1 = n :: \ell'_1$ with $(\ell'_1 ++ \ell_2) ++ \ell_3 = \ell'_1 ++ (\ell_2 ++ \ell_3)$ (IH).
We show: $((n :: \ell'_1) ++ \ell_2) ++ \ell_3 = (n :: \ell'_1) ++ (\ell_2 ++ \ell_3)$
This holds as
$$\begin{aligned} & ((n :: \ell'_1) ++ \ell_2) ++ \ell_3 \\ &= (n :: (\ell'_1 ++ \ell_2)) ++ \ell_3 \text{ by definition of } ++ \\ &= n :: ((\ell'_1 ++ \ell_2) ++ \ell_3) \text{ by definition of } ++ \\ &= n :: (\ell'_1 ++ (\ell_2 ++ \ell_3)) \text{ by IH} \\ &= (n :: \ell'_1) ++ (\ell_2 ++ \ell_3) \text{ by definition of } ++ \end{aligned}$$

Generalization/Strengthening

$$\forall c \ n. \text{repeat } n \ c \ + \ + \text{repeat } n \ c = \text{repeat } n \ (c + c)$$

```
Fixpoint repeat (n count : nat) : natlist :=  
  match count with  
  | 0 => nil  
  | S count' => n :: (repeat n count')  
end.
```

```
Fixpoint app (l1 l2 : natlist) : natlist :=  
  match l1 with  
  | nil => l2  
  | h :: t => h :: (app t l2)  
end.
```


Generalization/Strengthening

- To show: $\forall n \ c_1 \ c_2 . \text{repeat } n \ c_1 \ ++ \ \text{repeat } n \ c_2 = \text{repeat } n \ (c_1 + c_2)$
- Proof: By induction on c_1
 - Suppose that $c_1 = 0$.
We show: $\text{repeat } n \ 0 \ ++ \ \text{repeat } n \ c_2 = \text{repeat } n \ (0 + c_2)$
This holds as: $\text{repeat } n \ 0 \ ++ \ \text{repeat } n \ c_2 = [] \ ++ \ \text{repeat } n \ c_2 = \text{repeat } n \ c_2 = \text{repeat } n \ (0 + c_2)$
by definitions of $++$ and $+$
 - Suppose that $c_1 = S \ c'_1$ with $\text{repeat } n \ c'_1 \ ++ \ \text{repeat } n \ c_2 = \text{repeat } n \ (c'_1 + c_2)$ (IH).
We show: $\text{repeat } n \ (S \ c'_1) \ ++ \ \text{repeat } n \ c_2 = \text{repeat } n \ ((S \ c'_1) + c_2)$
This holds as
 $\text{repeat } n \ (S \ c'_1) \ ++ \ \text{repeat } n \ c_2$
 $= (n :: \text{repeat } n \ c'_1) \ ++ \ \text{repeat } n \ c_2$ by definition of repeat
 $= n :: (\text{repeat } n \ c'_1 \ ++ \ \text{repeat } n \ c_2)$ by definition of $++$
 $= n :: (\text{repeat } n \ (c'_1 + c_2))$ by IH
 $= \text{repeat } n \ (S \ (c'_1 + c_2))$ by definition of repeat
 $= \text{repeat } n \ ((S \ c'_1) + c_2)$ by definition of $+$

Stepping Back and Revising

$$\forall \ell . \text{length} (\text{rev } \ell) = \text{length } \ell$$

```
Fixpoint length (l:natlist) : nat :=  
  match l with  
  | nil => 0  
  | h :: t => S (length t)  
end.
```

```
Fixpoint rev (l:natlist) : natlist :=  
  match l with  
  | nil      => nil  
  | h :: t => rev t ++ [h]  
end.
```

Stepping Back + Revising

- To show: $\forall \ell . \text{length} (\text{rev } \ell) = \text{length } \ell$
- Proof: By induction on ℓ
 - Suppose that $\ell = []$.
We show: $\text{length} (\text{rev } []) = \text{length } []$
This holds as: $\text{length} (\text{rev } []) = \text{length } [] = \text{length } []$ by definition of rev
 - Suppose that $\ell = n :: \ell'$ with $\text{length} (\text{rev } \ell') = \text{length } \ell'$ (IH).
We show: $\text{length} (\text{rev } (n :: \ell')) = \text{length } (n :: \ell')$
This holds as
 $\text{length} (\text{rev } (n :: \ell'))$
 $= \text{length} (\text{rev } \ell' + + [n])$ by definition of rev
 $= 1 + \text{length} (\text{rev } \ell')$ by Lemma
 $= 1 + \text{length } \ell'$ by IH
 $= \text{length } (n :: \ell')$ by definition of length



Quiz

<https://etc.ch/HCCs>

Theorem `foo1` : forall n:nat, forall l:natlist,
repeat n 0 = l -> length l = 0.

```
Fixpoint repeat (n count : nat) : natlist :=  
  match count with  
  | 0 => nil  
  | S count' => n :: (repeat n count')  
end.
```

```
Fixpoint length (l:natlist) : nat :=  
  match l with  
  | nil => 0  
  | h :: t => S (length t)  
end.
```

Option Types

Partial Maps

Summary

Datatypes on top of Natural Numbers

- Pairs

```
Inductive natprod : Type :=  
  | pair (n1 n2 : nat).
```
- Lists

```
Inductive natlist : Type :=  
  | nil  
  | cons (n : nat) (l : natlist).
```
- Options

```
Inductive natoption : Type :=  
  | Some (n : nat)  
  | None.
```
- Partial Maps

```
Inductive partial_map : Type :=  
  | empty  
  | record (i : id) (v : nat) (m : partial_map).
```
- Trees

```
Inductive btree : Type :=  
  | leaf (n : nat)  
  | unary (n : nat) (t : btree)  
  | binary (n : nat) (t1 : btree) (t2 : btree).
```

Induction on Datatypes

```
induction l as [| n l' IHl']
```

```
induction t as  
[n | n t' IHt' | n t1' IHt1' t2' IHt2']
```

Generalising Statements

$$\forall c\ n. \text{repeat } c\ n\ ++\ \text{repeat } \cancel{c}\ n \\ = \text{repeat } (c + \cancel{c})\ n$$